

MOMENT SEARCH AT SCALE

One Index for Every Source

Turning a video-only moment search product into a multi-source knowledge engine: talks, research papers, and slide decks — one queue, one index, one answer with citations.

Assignment 3 · Architecture Deck

The Problem

- Existing system: **video-only**. Ingests YouTube talks, indexes frames + transcript, answers with timestamped citations.
- Real knowledge lives across **three kinds of source**: talks, papers, slide decks.
- Users want **one question, cited moments across every source** — the video timestamp *and* the paper page *and* the deck slide.

Two Queue Layers, Not One

Layer 1 — WFQ Scheduler (Postgres)

Sources sit `pending` in the manifest. A background thread admits at most `DISPATCH_MAX_INFLIGHT` at a time, round-robin across users, via an atomic `UPDATE ... WHERE status='pending' RETURNING .`

Stops one bulk uploader from starving everyone else.

Layer 2 — Prefect Cloud

Once admitted, the API fires a deployment run; workers long-poll over outbound HTTPS.

An orchestrator, not a broker — retries and a run dashboard. Fairness is entirely Layer 1's job.

The Core Design Decision

One shared vector index for every source.

- Video transcript chunks, paper pages, and deck slides all land in the **same text collection** — not three separate indexes.
- Retrieval is one nearest-neighbor query; results are fused and grouped by source afterward, not before.
- One query can surface a video moment, a paper page, and a deck slide side by side — they were never split apart to begin with.

- A new source kind = a new payload field, not a new index.

Locators Across Kinds

Video

Citation locator: `{start_ms, end_ms}` . Deep link seeks the player to the exact moment.

Paper

Citation locator: `{page}` . Deep link opens the PDF to the cited page.

Deck

Citation locator: `{slide}` . Deep link opens the deck to the cited slide.

Every citation carries `kind` + `locator` + `text` — grounded, not fabricated. Empty retrieval returns empty citations, never an invented one.

Resilience Is Application-Owned

- Ingestion checkpoints at every stage: parse → chunk → embed/upsert, each committed as a durable artifact before the status advances.
- A staleness sweep resets any source stuck mid-flight past a threshold back to `pending` — the dispatcher fairly re-admits it, and a new run resumes from the last committed checkpoint.
- Deterministic point IDs make redelivery safe: re-running embed/upsert never duplicates.
- We don't assume Prefect's own crash detection guarantees "0 dropped" — our own reconciler does.

Why a Queue at All

Search is latency-critical and read-only.

Ingestion is bursty and heavy — OCR, captioning, embedding hundreds of chunks.

Share a process and one backfill starves every user.

The queue makes the admin API trivial: validate, insert, enqueue, 202 —
while workers drain at their own pace, measured, not assumed.